RV32I 5-Stage Pipelined Core

Design document - rv32i_5stage_core - Abhijeet Jadhav - 2026-10-21

A. Project overview

A.1 Purpose

`rv32i_5stage_core` is a 5-stage, in-order RV32I processor core written in SystemVerilog. I wanted a core I could defend the way a design team would have to: every architectural effect of every retired instruction is checked against an instruction-set simulator I wrote separately, the pipeline-control rules are written down as assertions, the test environment is itself tested by putting deliberate bugs into the RTL, and the clock periods and cycle counts behind the performance figures come from tool reports kept next to the design.

A.2 Problem statement and motivation

A pipelined core is easy to get almost right. Forwarding, the load-use stall, branch flushing and trap handling all interact, and a core can pass a handful of hand-written programs while still hiding bugs that only show up at one particular register-dependence distance, or when a branch sits right behind a load. So I set myself two goals:

1. keep the hazard and control rules small enough that I can state them exactly, and
2. build a verification environment that notices when any one of those rules is broken, and show that it does by counting how many injected bugs it catches and which layer catches them.

I also wanted the work to be reproducible. Every number in this document comes from a command in the repository and is quoted from a result file, and where I didn't measure something I say so.

A.3 Scope and requirements

ID	Requirement	Status
R1	Implement the RV32I base ISA (all 37 user-level instructions; FENCE as a no-op; ECALL/EBREAK as halt)	implemented, checked against the ISS
R2	5-stage pipeline with full forwarding and a one-cycle load-use stall	implemented, checked
R3	Branch and jump handling with a selectable control-hazard policy (resolve in EX, or static prediction from ID)	implemented, both policies checked
R4	Byte, halfword and word loads and stores with the correct lane handling and sign/zero extension	implemented, checked
R5	A retirement-trace port so each instruction can be compared with a reference	implemented (RVFI subset)
R6	Trap handling for illegal encodings, misaligned accesses and misaligned branch / jump targets without corrupting architectural state	implemented as "stop and squash", checked (simulation and formal)
R7	Measure CPI, minimum clock period and area; compare against a single-cycle core built from the same blocks	measured (section H)
R8	Demonstrate that the verification environment can fail: injected-bug campaign	measured (section G)

I left some things out on purpose: CSRs and interrupts, the M and C extensions, privilege modes, caches and bus protocols, wait states, and unbounded formal proofs (the riscv-formal checks in section G.7 are bounded). Section I lists them as limitations.

A.4 Target application and concepts exercised

The core would suit an embedded controller that runs integer code out of tightly coupled memories. Building it meant dealing with pipeline hazards (RAW, load-use and control), operand forwarding, a write-first register-file bypass, static branch prediction, precise stop-on-trap behaviour and byte-lane handling in the load/store unit. Verifying it meant retirement tracing, lock-step co-simulation, constrained-random stimulus, assertions, coverage closure and mutation analysis. On the implementation side it went through logic synthesis and static timing analysis, and one design change was driven directly by the critical path.

A.5 Key features and measurable objectives

- All 37 RV32I instructions in a 5-stage pipeline, with a steady-state CPI of 1.0 on code without hazards.
- Two control-hazard policies selected by `BP_MODE`, and two ways of computing the forwarding selects, selected by `FWD_PRECOMP`.
- A retirement-trace port in the style of RVFI that a parameter (`RVFI_EN=0`) removes, so the synthesised core carries no verification hardware.
- Objective O1: every directed test, benchmark and random program matches the ISS instruction for instruction (pass/fail, section G).
- Objective O2: every injected bug is caught, and every assertion fires in at least one injected-bug run (section G).
- Objective O3: a measured CPI, minimum period and area for each configuration, and a measured execution-time comparison with a single-cycle core (section H).

B. Architecture and block diagram

B.1 Overview

The datapath is the classic five stages. Instructions go IF -> ID -> EX -> MEM -> WB through four pipeline registers (IF/ID, ID/EX, EX/MEM and MEM/WB). Each pipeline register has a valid bit. A bubble is simply a stage with `valid = 0` and every side-effect flag cleared, so it can never write a register, write memory, cause a hazard or redirect the fetch.

The core knows nothing about memory timing. The instruction port is a combinational read (`imem_addr` -> `imem_rdata`), and the data port is a combinational read with a synchronous, byte-strobed write. Both memories live in the testbench, or in whatever system the core is placed in. The architecture diagram is in section C, and the module descriptions below refer to it.

B.2 Modules and responsibilities

Module	File	Responsibility	Combinational / sequential
`rv32i_core`	`rtl/rv32i_core.sv`	Pipeline registers, PC, next-PC select, flush/stall/redirect control, halt state, RVFI trace	both
`rv32i_decode`	`rtl/rv32i_decode.sv`	Opcode decode, immediate generation, control signals, illegal/ECALL/EBREAK detection	combinational
`rv32i_regfile`	`rtl/rv32i_regfile.sv`	31 x 32-bit registers, two read ports, one write port, write-first bypass, x0 = 0	both
`rv32i_alu`	`rtl/rv32i_alu.sv`	add, sub, slt, sltu, and, or, xor, sll, srl, sra	combinational
`rv32i_branch`	`rtl/rv32i_branch.sv`	BEQ, BNE, BLT, BGE, BLTU, BGEU comparison	combinational
`rv32i_lsu`	`rtl/rv32i_lsu.sv`	Store byte strobes and lane replication; load lane select and sign/zero extension; misalignment detection	combinational
`rv32i_hazard`	`rtl/rv32i_hazard.sv`	Load-use stall; forwarding selects (EX-stage compare, or ID-stage precompute)	combinational
`rv32i_core_1cyc`	`rtl/rv32i_core_1cyc.sv`	Single-cycle baseline built from the same blocks (measurement only)	both
`tb_core`	`tb/tb_core.sv`	64 KiB unified memory model, trace writer, cycle accounting	testbench
`rv32i_checker`	`verif/rv32i_checker.sv`	Bound assertions and cover points	verification

B.3 Interfaces

Every `rv32i_core` signal is on the single clock `clk`, and `rst_n` is a synchronous active-low reset:

Signal	Dir	Width	Meaning
`clk`, `rst_n`	in	1	clock; synchronous active-low reset

Signal	Dir	Width	Meaning
imem_addr	out	32	address of the instruction to fetch (pc_f), word aligned
imem_rdata	in	32	instruction word, combinational response
dmem_addr	out	32	word-aligned data address
dmem_re	out	1	read enable (suppressed on a misaligned access)
dmem_we	out	4	byte strobes for a store (zero for no write; suppressed on a misaligned access)
dmem_wdata	out	32	store data, positioned in its byte lanes
dmem_rdata	in	32	data word, combinational response
halted	out	1	sticky: the core has stopped after ECALL/EBREAK/illegal/misaligned
rvfi_*	out	various	retirement trace (valid, order, insn, pc_rdata, pc_wdata, rs1/rs2 addr and data, rd addr and data, mem addr/rmask/wmask/rdata/wdata, trap, halt); zero when RVFI_EN=0

Parameters: RESET_PC (default 0), BP_MODE (0 or 1), FWD_PRECOMP (0 or 1), RVFI_EN (0 or 1).

B.4 Data paths, control paths and pipeline registers

Data path. imem_rdata -> IF/ID -> decode -> (register read) -> ID/EX -> forwarding muxes -> ALU -> EX/MEM -> load/store unit and dmem -> MEM/WB -> write-back select -> register file. Operand A of the ALU is rs1, the PC (for AUIPC) or zero (for LUI), and operand B is rs2 or the immediate. JAL and JALR write pc+4 instead of the ALU result.

Forwarding path. alu_result_m (EX/MEM) and wb_data_w (MEM/WB) feed the two forwarding muxes in front of the ALU and the branch comparator. A load in MEM is never used as a forwarding source, because its data isn't ready yet; the load-use stall makes sure no consumer is ever in EX right behind it.

Control path. stall_d from the hazard unit holds the PC and IF/ID and puts a bubble into ID/EX. redirect_ex (from EX) loads the PC from target_ex and flushes IF/ID and ID/EX. redirect_id (from ID, only with BP_MODE=1) loads the PC from target_id and flushes IF/ID. trap_ex and trap_mem squash the younger instructions and set the sticky halted flag, and after that no new instruction enters the pipeline.

Pipeline registers.

Register	Carries
IF/ID	valid_d, pc_d, instr_d
ID/EX	valid_e, control flags (reg_write, mem_read, mem_write, is_branch, is_jal, is_jalr, trap), pc_e, immediate, rs1/rs2 data and addresses, rd, ALU op and operand selects, funct3, predicted-taken bit, and (with FWD_PRECOMP=1) the registered forwarding selects
EX/MEM	valid_m, reg_write, mem_read, mem_write, ALU/link result, store data, rd, funct3
MEM/WB	register-write flag, write-back data (load data or result), rd

With RVFI_EN=1 the instruction word, next PC, source values, memory records and halt/trap flags also travel down the pipeline.

B.5 State machines

There's no explicit FSM. The only control state is the valid bit of each stage and the sticky halted flag. The stall and flush behaviour that an FSM would normally hold is a combinational function of what's in each stage (section E.3), and that is what makes it possible to write it down as assertions (section G.4).

C. Draw.io architecture diagram

The diagram is generated from a single specification by docs/diagrams/make_diagrams.py, using dgm.py, a small drawing helper of mine that sits next to it in docs/diagrams/. It writes the editable draw.io file and the images from the same objects, so the two can't drift apart. The diagram shows the five stages, the pipeline registers, the data flow in blue, the control flow in dashed orange, the two forwarding paths back into EX, and the register-write loop from WB to the register file.

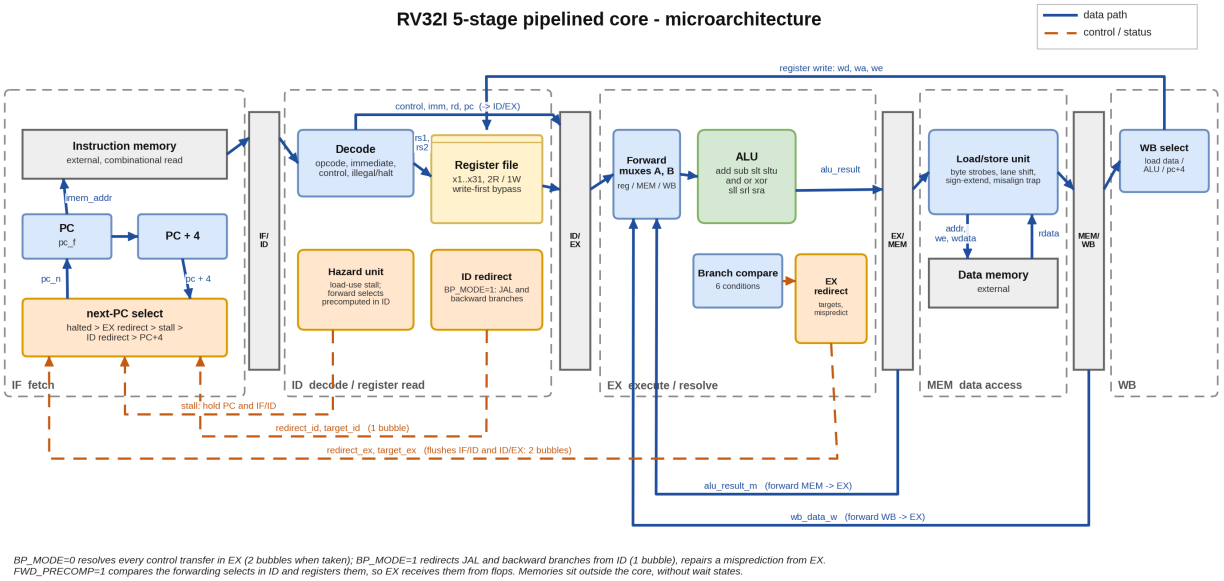


Figure C.1 - RV32I pipelined core: microarchitecture (data flow solid blue, control flow dashed orange)

The diagram files are in `docs/diagrams/`: `rv32i_architecture.drawio` (the editable source; it opens in diagrams.net or the draw.io desktop app), `rv32i_architecture.png` (a 2800 px export) and `rv32i_architecture.svg`, plus `rv32i_flowchart.*` and `rv32i_pipeline_timing.*`. The images are drawn with matplotlib using the same geometry as the `.drawio` file, so they aren't exports from the draw.io application.

D. Design flowchart

D.1 Operation from reset to retirement

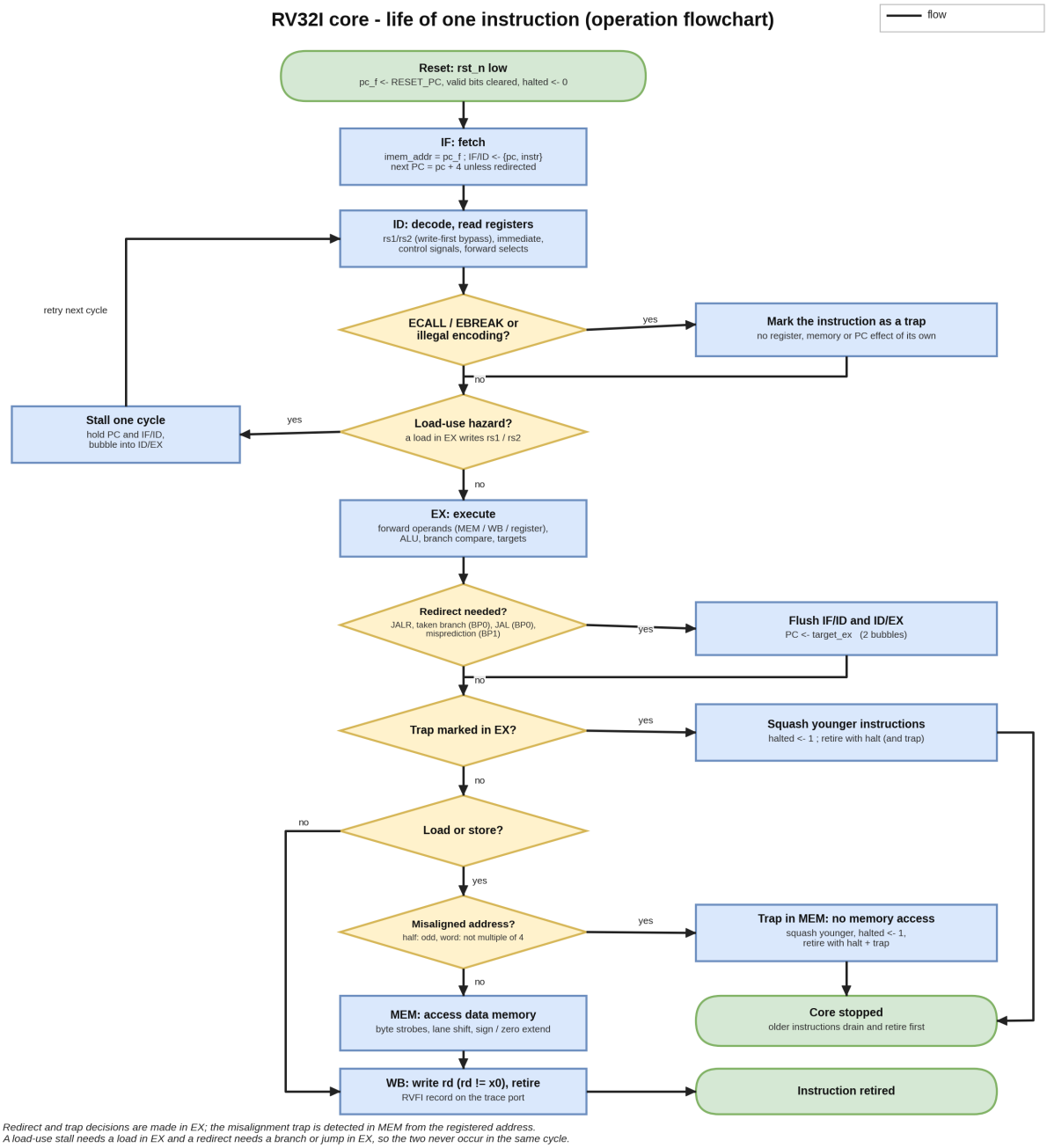


Figure D.1 - Life of one instruction: reset, fetch, decode, hazard check, execute, redirect, trap, memory, write-back

After reset, `pc_f` is loaded with `RESET_PC`, the valid bits and `halted` are cleared, and the core fetches continuously. In ID, an ECALL, EBREAK or illegal encoding is marked as a trap (it does nothing by itself), and a load-use hazard holds the instruction for one cycle. EX resolves control transfers and acts on a marked trap, MEM does the data access unless the address is misaligned, and WB writes the register and emits the retirement record. The two exceptional paths, a trap in EX and a misaligned access in MEM, end with the core stopped: older instructions drain and retire first, and younger ones are squashed.

D.2 Hazard timing example

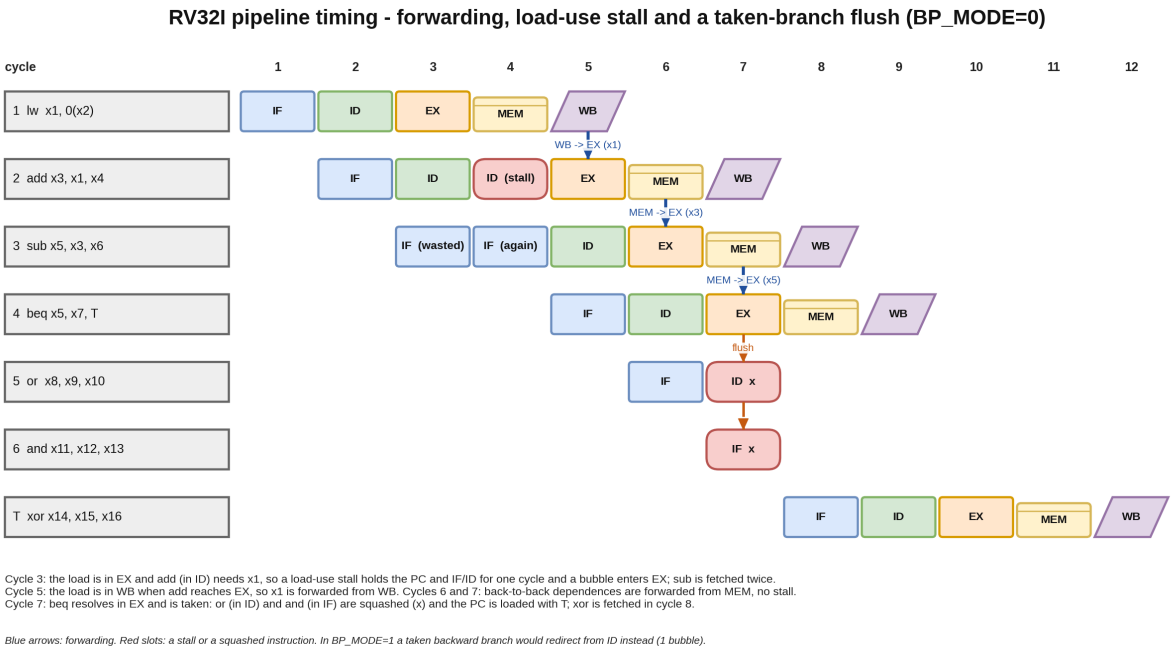


Figure D.2 - Forwarding, a load-use stall and a taken-branch flush, cycle by cycle (BP_MODE=0)

The example runs `lw x1,0(x2); add x3,x1,x4; sub x5,x3,x6; beq x5,x7,T; or; and; ...`. In cycle 3 the load is in EX and `add`, in ID, needs x1, so the PC and IF/ID hold for a cycle and a bubble goes into EX. In cycle 5 the load is in WB when `add` reaches EX, and x1 is forwarded from WB. `sub` and `beq` get their operands from MEM. In cycle 7 `beq` resolves as taken, the two younger instructions are squashed, and `xor` at the target is fetched in cycle 8.

E. Detailed design explanation

E.1 Step-by-step operation

1. **Fetch.** `imem_addr = pc_f`. On the clock edge IF/ID captures `{pc, instruction}` unless the PC and IF/ID are held for a stall or flushed.
2. **Decode.** `rv32i_decode` pulls out `rs1`, `rs2`, `rd`, the immediate (I, S, B, U or J form) and the control flags. The register file is read in the same cycle, and a write in that cycle to a register being read is passed straight through (write-first). That makes a RAW dependence three instructions apart free.
3. **Hazard check.** If the instruction in EX is a load whose destination is a register this instruction reads, this instruction stalls for one cycle.
4. **Execute.** Both operands go through the forwarding muxes, the ALU computes the result (or the branch comparator decides), and the target adders compute `pc+imm` and, for JALR, `(rs1+imm) & ~1`. Control transfers redirect the fetch according to `BP_MODE`.
5. **Memory.** Loads and stores go through `rv32i_lsu`. Stores drive byte strobes and a word with the data copied across its lanes, and loads select the addressed lane and extend it.
6. **Write-back.** The selected value is written to `rd` (unless `rd = x0`) and the RVFI record is emitted.

E.2 Forwarding, in detail

The rule is the same for operand A and operand B: the youngest older producer of the register wins. If the instruction in MEM writes `rs1` (and `rs1 != x0`), forward `alu_result_m`. Otherwise, if the instruction in WB writes `rs1`, forward `wb_data_w`. Otherwise use the value read from the register file in ID.

I implemented that rule in two ways:

- `FWD_PRECOMP=0` compares `rs1_e/rs2_e` with `rd_m/rd_w` in EX. The compares and the priority logic then sit on a path that carries on through the forwarding mux, the ALU or branch comparator and the redirect logic into the PC.
- `FWD_PRECOMP=1` does the comparison a stage earlier. While the instruction is in ID, the producer now in EX will be in MEM, and the one now in MEM will be in WB, by the time the instruction reaches EX. So the select can be computed from `rd_e`/`rd_m` and registered into ID/EX, and EX gets it straight from flops. A load-use stall puts in a bubble, and the select is recomputed on the next cycle, once the load has moved on.

The two versions are interchangeable, by construction and in testing: both pass every test, and the same assertions, written in terms of the EX-stage register contents, hold for both.

E.3 Stall, flush and redirect

Event	Detected in	Effect
Load-use hazard	ID (`stall_d`)	PC and IF/ID hold; bubble into ID/EX; the consumer re-enters EX one cycle later, with the load in WB
Taken branch / JAL (`BP_MODE=0`), JALR (both)	EX (`redirect_ex`)	PC <- `target_ex`; IF/ID and ID/EX flushed (2 bubbles)
JAL, backward branch (`BP_MODE=1`)	ID (`redirect_id`)	PC <- `target_id`; IF/ID flushed (1 bubble); mispredictions are repaired from EX (2 bubbles)
ECALL/EBREAK/illegal	EX (`trap_ex`)	younger instructions squashed, `halted` set
Taken branch, JAL or JALR whose target is not 4-byte aligned	EX (`misal_tgt_e`, folded into `trap_ex`)	no redirect, no link-register write, younger instructions squashed, `halted` set; a not-taken branch never traps. Found by the formal checks (G.7). With `BP_MODE=1` an ID redirect to a misaligned target is suppressed so that the PC never holds a misaligned value
Misaligned load/store	MEM (`trap_mem`)	memory access suppressed, younger instructions squashed, `halted` set

The next-PC select uses this priority: `halted` (hold), then `redirect_ex`, then `stall_d` (hold), then `redirect_id`, then `pc+4`. A load-use stall and an EX redirect can't happen in the same cycle, because the first needs a load in EX and the second a branch or jump there, and an assertion checks that no consumer slips past the stall.

E.4 Load/store unit

For stores the unit copies the byte or halfword across the whole word and sets the matching strobes (byte: `0001 << addr[1:0]`; halfword: `0011` or `1100`; word: `1111`), so the memory only has to honour the strobes. For loads it shifts the returned word right by `8 * addr[1:0]` and then sign- or zero-extends the byte or halfword. A halfword access at an odd address, or a word access at an address that isn't a multiple of 4, is misaligned: it isn't performed, and the instruction traps.

E.5 Reset, latency, throughput, back-pressure and errors

- **Reset.** Synchronous and active-low. `pc_f <- RESET_PC`, the valid bits of IF/ID, ID/EX, EX/MEM and MEM/WB are cleared, and `halted` is cleared. The register file isn't reset by default, since that would add a multiplexer to every flop; programs initialise what they read.
- **Latency.** The first instruction retires four cycles after reset is released, while the pipeline fills, so `cycles = 4 + instret + bubbles`.
- **Throughput.** One instruction per cycle when there are no hazards. A load-use pair costs one bubble. A taken control transfer costs two bubbles with `BP_MODE=0`, or one with `BP_MODE=1` when it's a correctly predicted backward branch or a JAL, and a misprediction costs two.
- **Back-pressure.** There isn't any, by design: the memories are assumed to answer in the same cycle. A real integration would need a global stall that holds every stage while a memory isn't ready, and that is on the future-work list.
- **Errors.** The only error behaviour is to stop and raise a flag (`halted`, `rvfi_trap`, `rvfi_halt`).

E.6 Key design decisions and trade-offs

Decision	Alternatives	Reason / trade-off
Resolve branches in EX (policy 0)	resolve in ID with extra forwarding paths	EX already has the forwarded operands; resolving in ID would need a comparator behind a second forwarding network
Static BTFN from ID (policy 1)	a dynamic predictor or BTB	the smallest change that removes a bubble from every loop iteration; the measured benefit is in section H
Precomputed forwarding selects	keep the compare in EX	moves the 5-bit compares and the priority logic off the path through the ALU or comparator into the PC, for the price of four flops and some compare logic in ID
Stop on trap	CSRs and a trap vector	keeps the architectural model small and exact, without pretending to implement what isn't there
RVFI port as a parameter	always present	lock-step checking needs it, but the area and timing figures shouldn't pay for it
Registers not reset	reset all of them	saves area; programs initialise their registers, and the ISS rejects any program that reads a register before writing it
Verilog-2005-friendly constructs (no structs, enums or packages)	SystemVerilog packages	the same source has to go through both Verilator 5.020 and Yosys 0.33

F. Interface and timing details

F.1 Clocking and reset

One clock domain, rising-edge flops and a synchronous reset. There's no clock-domain crossing.

F.2 Memory handshake

There's no valid/ready handshake: a read request and its response happen in the same cycle. These are the timing assumptions the surrounding system has to meet:

Port group	Assumption used for timing (constraints/rv32i.sdc)
`imem_rdata`, `dmem_rdata`	arrive 2.0 ns after the clock edge (memory clock-to-output)
`imem_addr`, `dmem_addr`, `dmem_re`, `dmem_we`, `dmem_wdata`, `halted`	must be stable 1.0 ns (0.5 ns for `halted`) before the next edge
`rst_n`	arrives 0.5 ns after the edge, driven by a `buf_2`

They are assumptions about the environment, the same for every configuration, and I repeat them next to every timing result.

F.3 Write timing and read-after-write through memory

A store writes the memory at the end of its MEM cycle, with the strobes valid during that cycle. A later load from the same address is at least one stage behind, so it sees the new data and no store-to-load forwarding is needed. The testbench ignores writes while reset is asserted, because the core's flops hold unspecified values until the first reset edge.

F.4 Supported conditions and limitations

Instruction fetch has to be 4-byte aligned: a taken branch, JAL or JALR to a target that isn't 4-byte aligned traps in EX (see E.3 and G.7). Data accesses have to stay within the testbench's 64 KiB, there are no wait states, and there's a single hart.

G. Verification and testing

G.1 Objectives and test plan

Objective	Evidence
Every architectural effect of every instruction is correct	lock-step with the ISS (trace and final memory)
The hazard and control rules hold exactly	bound assertions in both directions
The tests can fail	73 injected bugs
The rules hold for every input within a bound, not only for the test programs	riscv-formal bounded checks (G.7)
The stimulus reaches the interesting corners	named cover points and line coverage
Neither the ISS nor the RTL is the only source of truth	expected values computed independently in the test generators

G.2 Testbench architecture

`tb_core` instantiates the core (or the single-cycle baseline), a unified memory that both the instruction and the data port see, a trace writer and cycle counters. `tools/rvasm.py` assembles each program into a hex image, and the ISS `tools/rviss.py` runs the same image and writes its own trace. `tools/compare_trace.py` compares the two traces field by field (memory data only in the byte lanes the mask marks), and the runner then compares the final memory images and reads the program's own verdict from register `a0` in the dump. The bound assertions are on in every test.

G.3 Directed and random tests

The directed tests come from `tools/gen_directed.py`, which also computes their expected values:

Test	What it exercises
t01	every register-register ALU operation over a 10 x 10 operand grid including 0, 1, INT_MIN, INT_MAX, all-ones and shift amounts up to 31
t02	immediate forms, LUI, AUIPC
t03	all load/store sizes, every byte lane, sign and zero extension, neighbouring bytes untouched
t04	the six branches, taken and not taken, signed/unsigned boundaries, loops with trip counts 1 to 100, nested loops
t05	RAW dependences at distance 1 to 4 on each operand, forwarding priority (MEM over WB), x0 as destination and source, store address and data forwarding
t06	load followed by a consumer in every operand position, store data/address, branch operands, JALR target, pointer chasing, load into x0
t07	wrong-path squash after taken branches/JAL/JALR, link values, nested call/return, back-to-back branches
t08a-r	illegal encodings, CSR, misaligned loads/stores (memory must not change), EBREAK, ECALL squashing younger instructions, a trap behind a load-use stall; misaligned JAL / JALR / taken BEQ (forward and backward) targets (trap, no link write), a misaligned JALR right behind a load-use stall, and a not-taken branch with a misaligned offset (no trap)
t09, t10	FENCE and x0 behaviour; a backward branch whose operand comes straight from the preceding load (stall on a predicted-taken branch)

The benchmarks come from `tools/gen_bench.py`, and each result is checked against Python: Fibonacci, memcpy, bubble sort, sieve, a bitwise CRC-32 (checked against `zlib.crc32`), a linked-list walk, and a 6 x 6 matrix multiply that uses a software multiply and call/return.

The constrained-random programs come from `tools/rvgen.py`. The README lists its constraints: the instruction mix, the dependency and load-use bias, edge-value operands, forward branches over wrong-path filler, counted and memory-resident loops, JALR through computed addresses and the occasional trap ending.

Measured results (from `results/directed.txt`, `results/random.txt`, `results/cpi.md`):

Test class	Result
Directed tests	28/28 tests passed (BP_MODE 0,1)

Test class	Result
Constrained-random programs	200/200 tests passed (BP_MODE 0,1); 161597 instructions retired and compared
Benchmarks (self-checking, both BP modes)	all 7 pass; table in H.3

G.4 Assertions and properties

`verif/rv32i_checker.sv` is bound into the core and holds 48 labelled assertions, in these groups:

- **Forwarding**: `a_fwd_{a,b}_{mem,wb}_{only,always}` and `a_fwd_{a,b}_none` state the rule in both directions for valid instructions.
- **Load-use**: `a_stall_always` / `a_stall_only` (the stall is exactly the hazard), `a_stall_bubble`, `a_stall_holds_pc`, `a_stall_holds_id`, `a_no_load_use_slip` (a consumer is never in EX directly behind the load that feeds it).
- **Control flow**: `a_jalr_redirects`, `a_redirect_only`, `a_taken_redirects`, `a_nt_no_redirect`, `a_no_id_redirect`, `a_mispredict`, `a_predict_ok`, `a_jal_in_id`, `a_id_redirect_only`, `a_btfn`, `a_ex_flush_two`, `a_id_flush_one`, `a_ex_target`, `a_pc_aligned`, `a_misal_traps`, `a_misal_only`.
- **Traps**: `a_halted_sticky`, `a_halted_drains`, `a_trap_squashes`, `a_trap_mem_squash_m`, `a_no_retire_after_halt`, `a_trap_no_write`.
- **Side effects**: `a_rw_valid_m`, `a_mem_valid_m`, `a_we_only/always`, `a_re_only/always`, `a_store_strobe_lane`.
- **Trace**: `a_x0_no_write`, `a_order_counts`, `a_trap_implies_halt`.

Verilator 5.020 handles implication and `$past` but not `##` sequence delays or covergroups, so I wrote the functional coverage as named `cover property` points.

G.5 Coverage

Merged coverage of the directed tests, the benchmarks and random programs, both BP modes (`results/coverage_summary.txt`):

```
source               : build/cov/merged.dat
named cover bins     : 143 total, 143 hit, 0 zero
rtl/ line blocks     : 117 total, 115 covered, 2 excluded, 0 uncovered
rtl/ files with no line records : rv32i_core_1cyc.sv (procedural blocks only are recorded; these are covered by toggle coverage, the
    cover bins and the assertions, not by the line gate)
rtl/ branches        : 72 total, 59 hit (reported, not gated)
rtl/ toggle bits     : 3853 total, 3773 toggled, 80 unhit (reported, not gated)
```

G.6 Checking the checks: injected bugs

`scripts/mutants.py` applies each of 73 one-line RTL bugs (listed in `verif/mutants.py`) to a copy of the RTL, builds it with and without the bound assertions, and runs the directed tests and 40 random programs against it. The campaign fails if any bug gets past every layer, if an assertion that should trip doesn't, or if any assertion is never seen to fire.

Result of the campaign (`results/mutants.txt`, first lines):

```
mutants: 73, caught 73
test set per mutant: 28 directed + 40 random programs, lock-step vs ISS
caught ONLY by an assertion (the other layers missed them): M07, M29, M33, M49, M50, M53, M58, M59, M60, M61, M63, M72,
    M73
assertions defined: 48; fired in at least one mutant run: 48
assertions with no mutant bound: none
assertions that fired in no run: none
PROBLEMS: none
```

This campaign taught me more than anything else in the project:

- The first run showed a gap in the stimulus. A load-use stall on a backward branch that `BP_MODE=1` predicts taken survived every test. I added a directed test (`t10`) and a class of random programs that keep their loop counters in memory, and that bug is caught now.
- Some bugs are caught only by an assertion and by no other layer: performance-only bugs such as an unnecessary stall, and squash bugs whose only victim is an instruction that never retires. The result marks them as caught only by an assertion.
- A cover point written as `$past(stall_d) && valid_e` could never be hit, because the instruction after a stall is always a bubble. Coverage closure exposed it, and I changed it to `$past(stall_d, 2)`.

- One of the 48 assertions, `a_no_id_redirect` (there are no ID redirects in `BP_MODE=0`), is only violated by a redirect that is redundant rather than wrong, so it states design intent more than a functional requirement.
- Later, the formal checks (G.7) found a defect that all of the above had missed, because no test program ever jumped to a misaligned target. The fix came with six more injected bugs (M69 to M74), three assertions and six directed tests.

G.6.1 Simulation commands and tools

```
make check      # lint, self-tests, directed tests, benchmarks, 200 random programs, coverage
make mutants    # the injected-bug campaign
make formal     # riscv-formal bounded checks (G.7); needs SymbiYosys, z3 and riscv-formal under TC=... (default ../../_toolchain)
```

You need Verilator 5.020 or newer, Python 3 and GNU make; Yosys and OpenSTA for section H; and SymbiYosys 0.33, z3 and riscv-formal for `make formal`.

G.7 Formal verification (riscv-formal, bounded)

The core's RVFI port is connected to the riscv-formal harness (`formal/wrapper.sv`, with instruction memory and load data as free inputs) and checked with SymbiYosys and z3 for all four `BP_MODE` x `FWD_PRECOMP` variants. Each variant gets 43 checks: one per RV32I instruction (37), plus register-file consistency, PC consistency forward and backward, instruction uniqueness, causality and the illegal-instruction rule (6).

Result of the bounded checks on the current core (`results/formal.txt`):

```
riscv-formal bounded model checks of rv32i_core (RVFI_EN=1), SymbiYosys + z3, Yosys 0.33, isa rv32i, one retirement channel
instruction memory and load data are free inputs; reset is asserted in the first cycle; RISCV_FORMAL_ALIGNED_MEM
BOUNDED checks (the depth is the number of cycles from reset explored), not unbounded proofs. depth per check family: insn 12, reg
    14, pc_fwd 14, pc_bwd 14, unique 14, causal 14, ill 12

BP_MODE=0 FWD_PRECOMP=0 : 43 checks, 43 PASS, 0 not PASS
    causal 1/1, ill 1/1, insn 37/37, pc_bwd 1/1, pc_fwd 1/1, reg 1/1, unique 1/1
BP_MODE=0 FWD_PRECOMP=1 : 43 checks, 43 PASS, 0 not PASS
    causal 1/1, ill 1/1, insn 37/37, pc_bwd 1/1, pc_fwd 1/1, reg 1/1, unique 1/1
BP_MODE=1 FWD_PRECOMP=0 : 43 checks, 43 PASS, 0 not PASS
    causal 1/1, ill 1/1, insn 37/37, pc_bwd 1/1, pc_fwd 1/1, reg 1/1, unique 1/1
BP_MODE=1 FWD_PRECOMP=1 : 43 checks, 43 PASS, 0 not PASS
    causal 1/1, ill 1/1, insn 37/37, pc_bwd 1/1, pc_fwd 1/1, reg 1/1, unique 1/1

scope: 37 per-instruction checks (every RV32I instruction except FENCE / ECALL / EBREAK / CSR), plus reg (register x5 only), pc_fwd
    and pc_bwd (parameter 5),
  unique, causal (parameter 5) and ill (an all-zero word must trap). NOT RUN: liveness / hang, the memory-bus checks, CSR checks (no
      CSRs),
  and any unbounded (k-induction / PDR) proof; FENCE, ECALL and EBREAK are not covered by an instruction check.

all variants: 172 of 172 checks PASS
```

Result of the same checks on the core before the fix (`results/formal_as_found.txt`):

```
riscv-formal on the core before the misaligned-target fix (formal/as_found/rv32i_core.sv), BP_MODE=0 FWD_PRECOMP=1
BP_MODE=0 FWD_PRECOMP=1 : 43 checks, 35 PASS, 8 FAIL
failing checks:
  insn_beq_ch0  (FAIL 0 15)
  insn_bge_ch0  (FAIL 0 24)
  insn_bgeu_ch0  (FAIL 0 23)
  insn_blt_ch0  (FAIL 0 26)
  insn_bltu_ch0  (FAIL 0 24)
  insn_bne_ch0  (FAIL 0 21)
  insn_jal_ch0  (FAIL 0 21)
  insn_jalr_ch0  (FAIL 0 17)
failing assertion: rvfi_insn_check.sv line 196, assert(spec_trap == trap): the specification says the transfer traps, the core did
    not
```

What the first run found. On the core as it was, eight checks failed: all six conditional branches, JAL and JALR. (I ran the pre-fix core in one variant only, `BP_MODE=0 FWD_PRECOMP=1`.) The specification says that, without the C extension, a taken branch, JAL or JALR whose target isn't 4-byte aligned raises an instruction-address-misaligned exception, and my core simply jumped there. The ISS rejected such programs and an assertion flagged a misaligned PC, but no test program ever produced one, so simulation never saw the problem. The fix is in EX (`misal_tgt_e`): the transfer traps, writes no link register and doesn't redirect. I extended the ISS, the single-cycle baseline, the directed tests (`t08m` to `t08r`), the checker (`a_misal_traps`, `a_misal_only`, `a_trap_no_write`) and the mutant list (M69 to M74) to match. The pre-fix core is kept in `formal/as_found/`, and `formal/run_as_found.sh` reproduces the eight failures (`results/formal_as_found.txt`).

What this is and isn't. Bounded model checking explores every input sequence up to the stated depth from reset, which isn't the same as an unbounded proof. I didn't run the liveness checks, the memory-bus

checks, CSR checks (there are no CSRs), `reg` for registers other than x5, or checks for FENCE, ECALL and EBREAK. riscv-formal has no instruction check for those three; the directed tests and the ISS cover them.

G.8 Known failures and gaps

No test fails. The gaps: the formal checks are bounded (G.7), riscv-tests and riscv-arch-test aren't run because there's no RISC-V toolchain here, and the single-cycle baseline is verified in lock-step but has no bound assertions or coverage of its own.

H. Synthesis, timing and performance results

H.1 Method

Yosys 0.33 maps the core, with RVFI removed, to Sky130 HD. I compare two mappings: Yosys' default `abc -D`, which does no buffering, and a timing-driven ABC script (`scripts/abc_speed.script`: delay mapping, fan-out buffering and gate sizing). OpenSTA times each netlist with `constraints/rv32i.sdc`. The minimum period is `T_min = 10 ns - WNS(10 ns)`, and I check it by re-running STA at +/-2 % of T_min. All figures are pre-layout (ideal clock, no wires) at the typical corner (`tt_025C_1v80`).

RVFI trace port removed (RVFI_EN=0). Ideal clock, no wires, no placement. Memory budget in the SDC: read data 2.0 ns after the edge, address/write data 1.0 ns before the next edge (assumption, identical for all rows). `speed` = timing-driven ABC script, `default` = `abc -D 10000`.

core	mapping	cells	area (um^2)	flip-flops	T_min (ns)	F_max (MHz)	method check (+2 % meets, -2 % fails)
5-stage BP_MODE=0 FWD_PRECOMP=0	speed	6951	75452	1371	5.549	180.2	verified (WNS +0.111 / -0.111 ns)
5-stage BP_MODE=0 FWD_PRECOMP=1	speed	6906	75275	1365	4.330	230.9	verified (WNS +0.086 / -0.087 ns)
5-stage BP_MODE=1 FWD_PRECOMP=0	speed	7374	77986	1372	6.072	164.7	verified (WNS +0.122 / -0.121 ns)
5-stage BP_MODE=1 FWD_PRECOMP=1	speed	7396	77594	1366	4.842	206.5	verified (WNS +0.097 / -0.097 ns)
single-cycle	speed	6326	66975	1025	8.746	114.3	verified (WNS +0.175 / -0.175 ns)
5-stage BP_MODE=0 FWD_PRECOMP=1	default	6750	69276	1365	20.191	49.5	verified (WNS +0.404 / -0.404 ns)
5-stage BP_MODE=1 FWD_PRECOMP=1	default	7070	71540	1366	18.184	55.0	verified (WNS +0.364 / -0.364 ns)
single-cycle	default	7140	59023	1025	25.407	39.4	verified (WNS +0.508 / -0.508 ns)

H.2 Critical paths

- With `FWD_PRECOMP=0` the limiting path starts at the WB destination register, goes through the 5-bit compares and the forwarding select, takes the forwarded operand into the branch comparator, and runs through the redirect logic into the PC.
- With `FWD_PRECOMP=1` the select comes from a flop, and T_min drops accordingly (see the table above). The limiting path is then the forwarded operand through the ALU or comparator and the redirect logic.
- The limiting paths in the re-synthesised netlists, read from `build/ppa/*/timing.txt` and `synth.v` (start flop to end flop, with Yosys names), are: `BP_MODE=0 FWD_PRECOMP=0`: `u_hazard.rd_w[0]` to `pc_d[10]`; `BP_MODE=1 FWD_PRECOMP=0`: the same pair; `BP_MODE=0 FWD_PRECOMP=1`: `fwd_b_r[2]` (the registered forwarding select) to a flop named `u_decode.imm_j[17]` (a merged flop I didn't look into further); `BP_MODE=1 FWD_PRECOMP=1`: `fwd_b_r[2]` to `imem_addr[30]` (the PC register). The same configuration's T_min moved by up to 8 % between RTL revisions (6.024 to 5.549 ns, `results/ppa_before_misaligned_fix.md` against `results/ppa.md`). That shows how sensitive the mapping is to small RTL changes, so I don't draw conclusions from small differences between rows.

- Plain `abc -D` without buffering gives periods several times longer than the timing-driven mapping for the same RTL. That figure is dominated by high-fan-out nets rather than by the design, so every design conclusion here uses the timing-driven rows.

H.3 CPI and bubble sources

benchmark	instructions	BP_MODE 0 cycles	CPI	load-use stalls	EX redirects	BP_MODE 1 cycles	CPI	ID redirects	EX redirects	overlap	CPI change
b_bubblesort	13209	20123	1.523	1128	2891	17914	1.356	2353	634	48	-11.0 %
b_crc32	9684	14476	1.495	192	2298	13325	1.376	1730	954	193	-8.0 %
b_fib	522	710	1.360	0	92	621	1.190	92	2	1	-12.5 %
b_listwalk	712	1360	1.910	320	162	1201	1.687	162	2	1	-11.7 %
b_matmul	21644	31802	1.469	36	5059	27966	1.292	3968	1179	44	-12.1 %
b_memcpy	2894	4436	1.533	512	513	3928	1.357	514	3	2	-11.5 %
b_sieve	33604	51768	1.541	2044	8058	43861	1.305	8087	91	60	-15.3 %
all (sum)	82269	124675	1.515			108816	1.323				-12.7 %

Cycle accounting: accounted = 4 + instret + load-use stalls + 2 x EX redirects + ID redirects. BP_MODE 0: cycles == accounted (exact). BP_MODE 1: cycles <= accounted; `overlap` = accounted - cycles, slots squashed twice. Check: holds for all 14 runs

H.4 Execution time and the comparison with the single-cycle core

T_min (ns, pre-layout, timing-driven mapping): single-cycle = 8.746, 5-stage BP0, EX-compare fwd = 5.549, 5-stage BP0, ID-precomputed fwd = 4.330, 5-stage BP1, ID-precomputed fwd = 4.842

Execution time in microseconds (cycles x T_min). Lower is better. Speed-up is relative to the single-cycle core.

benchmark	instructions	single-cycle (us)	5-stage BP0, EX-compare fwd (us)	5-stage BP0, ID-precomputed fwd (us)	5-stage BP1, ID-precomputed fwd (us)	best 5-stage speed-up
b_bubblesort	13209	115.5	111.7	87.1	86.7	1.33x
b_crc32	9684	84.7	80.3	62.7	64.5	1.35x
b_fib	522	4.6	3.9	3.1	3.0	1.52x
b_listwalk	712	6.2	7.5	5.9	5.8	1.07x
b_matmul	21644	189.3	176.5	137.7	135.4	1.40x
b_memcpy	2894	25.3	24.6	19.2	19.0	1.33x
b_sieve	33604	293.9	287.3	224.2	212.4	1.38x
all	82269	719.5	691.8	539.8	526.9	**1.37x**

Speed-up of each 5-stage configuration over the single-cycle core, all benchmarks together: 5-stage BP0, EX-compare fwd 1.04x, 5-stage BP0, ID-precomputed fwd 1.33x, 5-stage BP1, ID-precomputed fwd 1.37x

Read with: pre-layout, ideal memories (2.0 ns read / 1.0 ns write budget assumed), 7 small kernels (82269 instructions), not a standard benchmark suite. The single-cycle figure is optimistic (see the script header), so these speed-ups are lower bounds under this model.

H.5 Measured, estimated, and not measured

Item	Status
Cycle counts, CPI, bubble sources	measured in RTL simulation
Minimum period, area, cell and flop counts	measured by Yosys + OpenSTA (pre-layout)
Execution time	derived: measured cycles x measured T_min
Routed setup / hold slack, router DRC count, area after place and route (one configuration, nine target periods)	measured by OpenROAD (H.6)
Frequency in silicon, signoff DRC / LVS, power with a real workload, routed results for the other configurations	NOT MEASURED
Behaviour with real memories (wait states, caches)	NOT MEASURED

H.6 Place and route (OpenROAD)

I placed and routed the `BP_MODE=1`, `FWD_PRECOMP=1` core, the configuration with the shortest execution time in H.4, with OpenROAD on Sky130 HD at nine target clock periods (`pnr/`, `make pnr`). The flow is the one I used for `alu_fifo` (floorplan, global placement, repair, clock-tree synthesis, repair_timing, global and detailed routing, antenna repair, SPEF extraction and a final STA on the extracted parasitics), with the same settings, which I didn't tune for this design. `pnr/pnr_report.py` read back and checked every run before I used its numbers.

Flow: floorplan at the stated utilisation, global placement, repair_design, clock-tree synthesis, repair_timing, global and detailed routing, antenna repair, RC extraction (SPEF), final STA with the propagated clock on the extracted parasitics (`pnr/pnr_flow.tcl`, adapted from alu_fifo). Netlist: the timing-driven Yosys mapping of `make ppa`. Constraints: `constraints/rv32i.sdc` (memory read 2.0 ns after the edge, address/write data 1.0 ns before the next edge: an assumption). Typical corner only; no hold fixing beyond what repair_timing did; no signoff DRC/LVS (Magic/KLayout/Netgen are not installed), so "DRC" below is the router's own count. Power is OpenSTA's default-activity figure, not a workload figure.

run	target period (ns)	setup WNS (ns)	hold WNS (ns)	TNS (ns)	route DRC	antenna viol.	slew / cap / fanout viol.	design area (um^2)	die (um)	checks
bp1_fwd1_p10.0	10.0	+0.626	+0.091	0.000	0	0	61 / 5 / 0	79330	450 x 450	all passed
bp1_fwd1_p6.0	6.0	-0.575	-0.131	-45.888	0	0	16 / 5 / 0	81775	450 x 450	all passed
bp1_fwd1_p7.0	7.0	-0.554	+0.117	-27.181	0	0	21 / 3 / 0	79922	450 x 450	all passed
bp1_fwd1_p8.0	8.0	-0.303	+0.093	-5.719	0	0	17 / 7 / 0	79534	450 x 450	all passed
bp1_fwd1_p8.4	8.4	-0.538	+0.073	-33.977	0	0	26 / 8 / 0	79420	450 x 450	all passed
bp1_fwd1_p8.6	8.6	-0.774	+0.091	-23.009	0	0	61 / 5 / 0	79330	450 x 450	all passed
bp1_fwd1_p8.8	8.8	-0.574	+0.091	-6.151	0	0	61 / 5 / 0	79330	450 x 450	all passed
bp1_fwd1_p9.0	9.0	-0.374	+0.091	-1.313	0	0	61 / 5 / 0	79330	450 x 450	all passed
bp1_fwd1_p9.5	9.5	+0.126	+0.091	0.000	0	0	61 / 5 / 0	79330	450 x 450	all passed

Worst routed setup path per run (start flop output net -> end flop input net, names from the post-repair netlist):

- `bp1_fwd1_p10.0`: `b_imm_e` -> `result_e[27]` (setup slack +0.626 ns)
- `bp1_fwd1_p6.0`: `\u_regfile.wd [29]` -> `_00363_` (setup slack -0.575 ns)
- `bp1_fwd1_p7.0`: `\u_regfile.wd [4]` -> `_00785_` (setup slack -0.554 ns)
- `bp1_fwd1_p8.0`: `b_imm_e` -> `_00789_` (setup slack -0.303 ns)

- `bp1_fwd1_p8.4`: `fwd_a_r[0]` -> `_00332_` (setup slack -0.538 ns)
- `bp1_fwd1_p8.6`: `b_imm_e` -> `result_e[27]` (setup slack -0.774 ns)
- `bp1_fwd1_p8.8`: `b_imm_e` -> `result_e[27]` (setup slack -0.574 ns)
- `bp1_fwd1_p9.0`: `b_imm_e` -> `result_e[27]` (setup slack -0.374 ns)
- `bp1_fwd1_p9.5`: `b_imm_e` -> `result_e[27]` (setup slack +0.126 ns)

Clock-tree sinks equal the flip-flop count in every accepted run; the netlist checksum matches the one recorded at the start of the run.

Smallest tried target that closed setup and hold in the routed, extracted design with zero route DRC violations: 9.5 ns (105.3 MHz), run `bp1_fwd1_p9.5`. This is the best of the targets that were run (a tested value, not an optimum): finer steps below it were not tried. The pre-layout minimum period of the same configuration is 4.842 ns (`results/ppa.md`); the smallest closing routed target is 1.96x that (derived: 9.5 / 4.842). Selected run: bp1_fwd1_p9.5 | target 9.5 ns | 105.3 MHz | setup WNS +0.126 ns | hold WNS +0.091 ns | route DRC 0 | antenna violations 0 | area 79330 um^2 In that run the routed design still has 61 max-slew, 5 max-capacitance and 0 max-fanout violating pins/nets (`check_types_final.rpt`; 9 of the violating pins are antenna diodes). Power, default activity (W): 9.90e-03. Design area before fillers: 79330 um^2 at 41.2 % utilisation. Clock skew report: Clock clk 1.216 source latency _11552_/CLK ^ -1.345 target latency _11599_/CLK ^ -0.130 setup skew

Runs bp1_fwd1_p8.6, bp1_fwd1_p8.8, bp1_fwd1_p9.0, bp1_fwd1_p9.5, bp1_fwd1_p10.0 have identical area and violation counts (repair_timing changed nothing at those targets) and their setup slack moves one for one with the period: the routed critical path of that layout is 9.374 ns (target minus slack, derived).

Run settings: core utilisation 40 %, repair_setup_margin 0.3 ns, global placement density 0.60 (alu_fifo's values, not tuned for this design); antenna repair between global and detailed routing: bp1_fwd1_p10.0 off, bp1_fwd1_p6.0 off, bp1_fwd1_p7.0 off, bp1_fwd1_p8.0 off, bp1_fwd1_p8.4 off, bp1_fwd1_p8.6 off, bp1_fwd1_p8.8 off, bp1_fwd1_p9.0 off, bp1_fwd1_p9.5 off.

What the table says, and what it doesn't:

- The routed design closes setup and hold with zero router DRC violations from 9.5 ns upward, and at none of the shorter targets I tried. The pre-layout minimum period of the same configuration is 4.842 ns (ideal clock, no wires), so the routed figure is about twice the pre-layout one. I didn't break down how much of that comes from clock-tree insertion delay and skew (the clock reports show about 1.2 to 1.4 ns of latency and 0.13 to 0.17 ns of skew), how much from wire load, and how much from the choices the repair step made.
- The results aren't monotonic in the target period. The 8.0 ns run ended 0.303 ns short while the 8.4 ns run ended 0.538 ns short, and from 8.6 ns upward `repair_timing` changed nothing, so those layouts are identical and only the period moves the slack. 9.5 ns is a value I tested, not an optimum, and I didn't try steps finer than the ones listed.
- The shortest target, 6.0 ns, also ends with a hold violation (-0.131 ns). Its worst hold path starts at the `rst_n` input, whose SDC input delay is 0.5 ns, and ends at a flop whose clock arrives 1.33 ns after the clock port. The SDC gives inputs no allowance for clock latency, so this is partly an artefact of the constraints rather than a problem in the design.
- Every routed run still has max-slew and max-capacitance violations: 61 and 5 in the 9.5 ns run, 9 of the 66 being antenna-diode pins, and the worst slew is 0.398 ns over the limit, on a net that drives mux select pins. I reported them but didn't fix them.
- I only routed this one configuration. The other three pipelined configurations and the single-cycle baseline weren't routed, so I don't claim a routed speed-up, and the speed-up in H.4 stays a pre-layout figure.
- Signoff DRC and LVS were NOT RUN, because Magic, KLayout and Netgen aren't installed. "DRC" here means the router's own count, and the power figure is OpenSTA's default-activity estimate for the extracted design, not a measurement on a workload.

- Two of the first three runs stopped with an OpenROAD error (GRT-0183) in the antenna repair between global and detailed routing. The runs reported here skip that step (`PNR_GRT_ANTENNA=0`); the antenna repair after detailed routing still ran, and there were 0 antenna violations at the end. The logs of the failed runs are in `results/pnr_grt_antenna/`. I also damaged one run myself, by editing the flow script while OpenROAD was still reading it. Its log is in `results/pnr_aborted/`, and I reran it.

I. Results, limitations and future improvements

I.1 Summary

The core implements all of RV32I. It passes every directed test, benchmark and random program in lock-step with an independent ISS, all 48 assertions hold, its cover points are closed, all 73 injected bugs are caught, and it passes 172 of 172 bounded riscv-formal checks across its four configurations (G.7). The measured gain of the pipeline over the single-cycle core built from the same RTL is modest (see the table in H.4), because a pipelined core is only as fast as its critical path. With the forwarding compare in EX, the pipeline came out at 0.92x of the single-cycle baseline when I first measured it and 1.04x after the misaligned-target fix (`results/performance_before_misaligned_fix.md`, `results/performance.md`). That isn't clearly better, and since the change between the two runs came with a fix that added logic, it's variation in the mapping. With the compare moved out of EX, the pipeline measured 1.27x and then 1.33x with `BP_MODE=0`, and 1.37x both times with `BP_MODE=1`.

I.2 Most meaningful engineering outcomes

1. A measured performance comparison, including the result that doesn't flatter the design: the straightforward pipeline didn't clearly beat the baseline (0.92x, then 1.04x after re-synthesis), while a change aimed at the critical path gave 1.33x to 1.37x. I kept and verified both variants.
2. A verification environment that can fail, shown with an injected-bug campaign whose gaps I found and closed.
3. A reference model and an assembler written for the project, so none of the evidence depends on a RISC-V toolchain.

I.3 Limitations

No CSRs, interrupts, or M and C extensions; the core stops on a trap; the memories are ideal, with no wait states; branch prediction is static only; timing is pre-layout apart from the one routed configuration, and there are no signoff checks; the formal checks are bounded, not proofs; and the single-cycle comparison is optimistic for the baseline (section H).

I.4 Future improvements

- A global stall for memory wait states, followed by an instruction cache and a data cache (my `mlc_cache_controller` project would be the natural partner).
- Zicsr, machine-mode traps and interrupts, then the M extension (a pipelined multiplier and an iterative divider).
- Dynamic branch prediction with a small BTB, measured with the same benchmarks and the same cycle accounting.
- Unbounded formal proofs (k-induction or PDR) of the riscv-formal properties, the liveness checks, and the riscv-arch-test suite once a RISC-V toolchain is available.
- Routing the other three pipelined configurations and the single-cycle baseline with the same flow, fixing the remaining max-slew violations, and running signoff DRC and LVS once Magic, KLayout and Netgen are available.

I.5 Interview talking points

- Why the load-use stall exists, why a load in MEM is never a forwarding source, and the assertion that checks it.
- Why MEM has to beat WB in the forwarding priority, and the test (`t05`) and mutants (M03, M43) that show it.

- How precomputing the forwarding selects in ID changes the critical path, what it costs (four flops), and the numbers before and after.
- What a taken branch costs under each policy, how I used the cycle-accounting identity to check the counters, and why it becomes an inequality with `BP_MODE=1`.
- How the lock-step trace works (an RVFI subset), and why the memory lanes are compared under the byte mask.
- Why the mutation campaign was worth running: the hole around a stalled predicted-taken branch, the unreachable cover point, and the bugs only an assertion caught.
- What I don't claim: no silicon frequency, no power figure, and a speed-up that is a lower bound against an optimistic baseline.

J. References and reproducibility

J.1 References

- *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA* (RV32I base), RISC-V International.
- RISC-V Formal Interface (RVFI) specification, in the riscv-formal project documentation.
- D. Patterson and J. Hennessy, *Computer Organization and Design, RISC-V Edition*, pipelining chapter (hazards, forwarding, branch handling).
- Verilator documentation (assertions, coverage); Yosys manual (`synth`, `abc`, `dfflibmap`); OpenSTA documentation.
- SkyWater SKY130 high-density standard-cell library (`sky130_fd_sc_hd`), typical corner liberty.

J.2 Directory structure

```
rv32i_5stage_core/
  rtl/        rv32i_defs.vh alu branch regfile decode lsu hazard core core_1cyc
  tb/         tb_core.sv
  verif/      rv32i_checker.sv checker_binds.sv mutants.py cov_exclusions.txt sim_main.cpp
  tools/      rvasm.py rviss.py rvgen.py compare_trace.py run_tests.py gen_directed.py gen_bench.py selftest_asm.py
  tests/      asm/ (28 directed)  bench/ (7 benchmarks)
  scripts/    build_sim.sh mutants.py cov_run.py ppa.py fmax.py cpi_report.py perf_report.py ...
  formal/     wrapper.sv checks.cfg.in run_formal.sh summarize.py run_as_found.sh as_found/
  pnr/        pnr_flow.tcl run_pnr.sh pnr_report.py config.tcl helpers.tcl flow_helpers.tcl
  constraints/ rv32i.sdc      sky130hd/ (liberty + LEF / tech files)      results/      docs/
```

J.3 Reproducing the results

```
make lint selftest        # structure and tool self-tests
make test bench rand      # lock-step results, CPI table
make cov                  # coverage
make mutants              # injected-bug campaign (about 15 minutes)
make ppa perf             # synthesis, timing, execution time
make formal               # riscv-formal bounded checks (about 10 minutes; needs SymbiYosys, z3, riscv-formal)
make pnr                  # OpenROAD place and route at nine target periods (about 40 minutes; needs make ppa first)
```

The reported results were produced with Verilator 5.020, Yosys 0.33, OpenSTA, OpenROAD v2.0, SymbiYosys 0.33 with z3 and riscv-formal, and Python 3.13. The source is at https://github.com/AbhijeetJ-512/rv32i-core.